Activity 1: Translate Requirements into a Verified ASP.NET Core Scaffold

Course: Build ASP.NET Web Apps with Vibe Coding (TGS-2023040477)

Alignment: LO1 · K1/A1

Duration: 55 minutes

Runnable snapshot: src/LocalCRM

Goal

Turn a LocalCRM customer-contact brief into component boundaries and a reproducible .NET 10 Razor Pages project.

What you will produce

requirements.md, component-map.md, pinned SDK file and a clean net10.0 Razor Pages scaffold

Safety and data boundary

Use only the supplied fictional records in mock-data/. Do not paste personal, customer, credential, or production data into an AI tool. Review every generated diff before running it.

Vibe script

> Act as a software-design assistant. Convert the supplied LocalCRM brief into UI, application-service, data and hosting components. Preserve requirement IDs. Do not write code yet. Return assumptions separately.

Step-by-step

Step 1 — Create a clean workspace

Create the cumulative project folder used throughout all ten Activities.

```
mkdir -p localcrm && cd localcrm && git init
```

Step 2 — Record the functional brief

Create requirements.md with REQ-001 create a contact, REQ-002 list contacts, REQ-003 accessible form, and REQ-004 expose a JSON API.

Create or update requirements.md:

```
# LocalCRM requirements

- REQ-001: Staff can create a contact with name, email and lifecycle stage.
- REQ-002: Staff can list saved contacts.
- REQ-003: The form is keyboard-operable and exposes labels/errors.
- REQ-004: An HTTP JSON API exposes create, list and item retrieval.
```

Step 3 — Ask the AI for boundaries

Use the Activity prompt. Save the response in component-map.md; check that every component maps to at least one REQ ID.

Step 4 — Pin the SDK

Confirm the installed .NET 10 SDK, then create global.json.

```
dotnet --list-sdks
dotnet new globaljson --sdk-version 10.0.100 --roll-forward latestPatch
```

Step 5 — Preview the template

Inspect the files before materialising them.

```
dotnet new webapp -n LocalCRM --framework net10.0 --dry-run
```

Step 6 — Create and build

Create the Razor Pages app and compile it.

```
dotnet new webapp -n LocalCRM --framework net10.0
dotnet build LocalCRM/LocalCRM.csproj
```

Expected evidence: Build succeeded with 0 errors.

Step 7 — Run and probe

Start the app on a known local URL in one terminal; probe from another.

```
dotnet run --project LocalCRM --urls http://localhost:5050
curl -i http://localhost:5050/
```

Expected evidence: HTTP/1.1 200 and an HTML Content-Type header.

Step 8 — Checkpoint

Exclude generated outputs, inspect the diff and commit the verified scaffold.

```
printf 'bin/\nobj/\n*.db\n' > .gitignore
git status --short
git add requirements.md component-map.md global.json .gitignore LocalCRM
git commit -m 'feat: scaffold LocalCRM from requirements'
```

Run the supplied snapshot

The Activity folder already contains a complete stage-appropriate ASP.NET Core project and mock data. From this Activity folder:

```
cd src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5101
```

Open http://localhost:5101. For Activities 7–10, also probe http://localhost:5101/api/contacts; for Activities 8–10, probe /openapi/v1.json and /health/ready.

Acceptance checklist

□ The project targets net10.0 and restores without an unreviewed package.

□ The app starts from this Activity folder without relying on another Activity.

□ The supplied mock contacts are fictional and load as described.

□ Positive and negative paths match the expected status or UI state.

□ The diff contains only intended files and no secret.

Troubleshooting

- NETSDK1045: install a .NET 10 SDK or select the installed 10.0 feature band in global.json.
- Address already in use: stop the previous Activity process or use --urls http://localhost:PORT with a free port.
- HTTP 404: compare the exact route, method, route constraint and Activity stage.
- HTTP 400/415: inspect ProblemDetails and confirm JSON shape plus Content-Type: application/json.

- SQLite failure (Activities 8–10): remove only the local training localcrm.db, then restart so the snapshot recreates mock data.

Reflection and evidence record

1. Explain how the evidence demonstrates LO1 and K1/A1.

2. Name one AI suggestion you rejected or changed and why.

3. Identify one negative path you tested and the observable result.

4. State the next improvement you would make without widening the approved scope.

Evidence item	Record the path, command result or observation
Build or test result	
Runtime URL and HTTP status	
Positive-path evidence	
Negative-path evidence	
Reviewed AI change or rejection	
Commit or evidence reference	